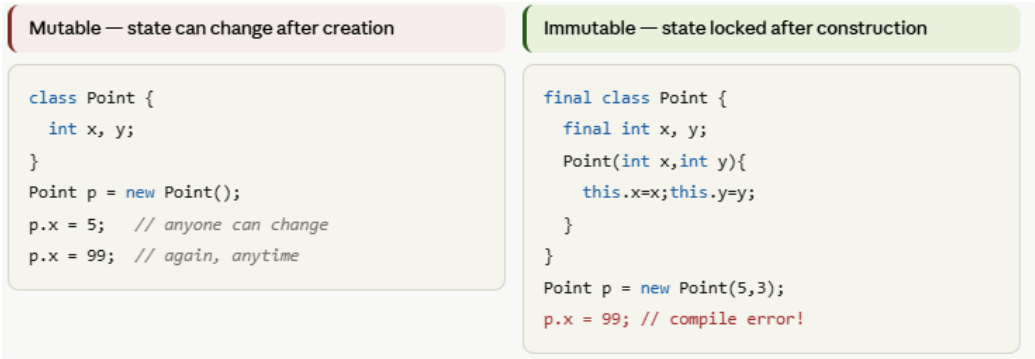
Immutabilité

What is immutability?



✓ Thread-safe by default — no synchronization needed

✓ Safe to share freely — no defensive copying for caller

✓ Perfect HashMap key — hashCode never changes

✓ Easier to reason about — state is always predictable

✓ Caching friendly — String pool, Integer cache

> "An immutable object's state cannot change after construction — it is inherently thread-safe and freely shareable."

The 5 rules to make a class truly immutable

```java
public final class Money {       // RULE 1: final class — no subclassing

    private final double amount;  // RULE 2: all fields private + final
    private final String currency;

    public Money(double a, String c) {
        this.amount   = a;          // RULE 3: assign only in constructor
        this.currency = c;
    }
                                    // RULE 4: no setters — ever
    public double getAmount()   { return amount; }
    public String getCurrency() { return currency; }

    // RULE 5: return NEW object instead of mutating
    public Money add(double extra) {
        return new Money(amount + extra, currency);
    }
}

Money m1 = new Money(100, "EUR");
Money m2 = m1.add(50);
```

```
// m1 → still 100 EUR — untouched
// m2 → new object 150 EUR
```

FC · PF · CO · NS · RN

Final Class · Private Final · Constructor Only · No Setters · Return New

Most common interview trap — mutable object inside

```
public final class Team {
    private final String       name;
    private final List<String> members; // final ref, mutable content!

    public Team(String name, List<String> members) {
        this.name    = name;
        this.members = members;      // stores the original list — WRONG
    }

    public List<String> getMembers() {
        return members;              // exposes mutable list — WRONG
    }
}

List<String> list = new ArrayList<>(List.of("Ali", "Sara"));
Team team = new Team("Dev", list);
list.add("Hacker");                // pollutes team.members!
team.getMembers().add("Hacker2"); // also works — object broken!
```

Fix — defensive copy in AND out

```
public final class Team {
    private final String       name;
    private final List<String> members;

    public Team(String name, List<String> members) {
        this.name    = name;
        this.members = List.copyOf(members); // copy on input ✓
    }

    public List<String> getMembers() {
        return List.copyOf(members);        // copy on output ✓
    }
}

list.add("Hacker");               // ignored — team has its own copy
team.getMembers().add("Hacker2"); // UnsupportedOperationException!
```

"final on a reference prevents reassigning the pointer — it does NOT prevent mutating the object it points to. That's why defensive copying is required."

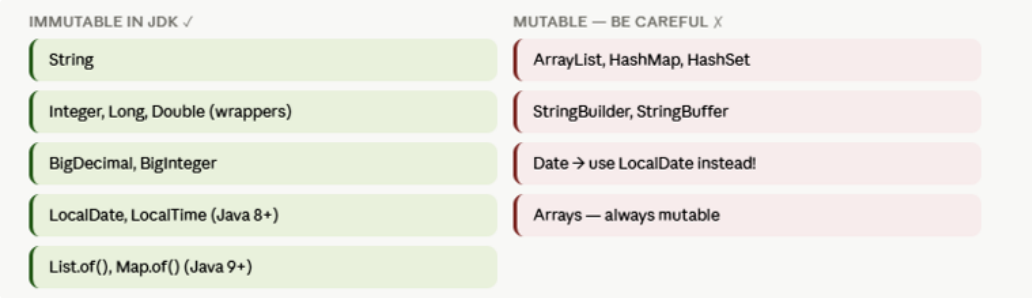
Famous immutable classes

```
// String — immutable since Java 1.0
String s = "hello";
s.toUpperCase();     // returns NEW String — s unchanged
s.replace("h","j");  // returns NEW String
System.out.println(s); // still "hello"

// String pool — safe because String is immutable
```

```java
String a = "Java";
String b = "Java";
a == b; // true — same object in pool

// Integer cache — -128 to 127
Integer x = 100; Integer y = 100;
x == y; // true — cached
Integer p = 200; Integer q = 200;
p == q; // false — outside cache range!
```



Integer est immutable — sa valeur ne peut pas changer après création. Le cache -128/127 est une optimisation mémoire : Java réutilise le même objet pour éviter d'en créer des milliers. C'est pourquoi `==` retourne `true` dans ce range — même objet — mais c'est un détail d'implémentation, pas une règle d'immutabilité.**Integer est TOUJOURS immutable** — peu importe la valeur

Ce que le cache -128/127 change, c'est **l'identité de l'objet** — pas l'immutabilité

```java
// Dans le cache — MÊME objet en mémoire
Integer x = 100;
Integer y = 100;
x == y;      // true  → même référence (objet réutilisé)
x.equals(y); // true  → même valeur

// Hors cache — deux objets DIFFÉRENTS en mémoire
Integer p = 200;
Integer q = 200;
p == q;      // false → deux objets distincts
p.equals(q); // true  → même valeur
```

Java 16 Records — immutability built-in

```java
// Record = immutable class, zero boilerplate
record Point(int x, int y) {}

// Compiler generates automatically:
public final class Point {           // final class ✓
    private final int x, y;          // private final ✓
    public Point(int x, int y) { ... } // constructor ✓
    public int x() { return x; }     // accessors ✓
    public int y() { return y; }
    // + equals, hashCode, toString ✓
}
```

⚠️ Record trap — same List problem applies!

```java
record Team(String name, List<String> members) {}
```

```
2
3  Team t = new Team("Dev", new ArrayList<>(List.of("Ali")));
4  t.members().add("Hacker"); // works! record NOT deeply immutable
5
6  // Fix — compact constructor
7  record Team(String name, List<String> members) {
8      Team {                                  // compact constructor
9          members = List.copyOf(members); // seal the list ✓
10     }
11 }
12 t.members().add("Hacker"); // UnsupportedOperationException — safe!
```

Records handle *shallow* immutability automatically. Deep immutability (mutable fields) is still YOUR responsibility.

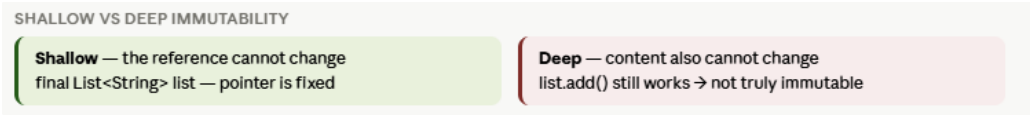
Why immutability = automatic thread safety

```
1  // Mutable shared state — race condition
2  class Counter {
3      int count = 0;
4      void increment() { count++; } // NOT thread-safe
5  }
6  // Thread A reads 5, Thread B reads 5
7  // Thread A writes 6, Thread B writes 6
8  // Expected 7 — Got 6 — silent data loss!
```

```
1  // Immutable — no shared mutable state = no race condition
2  final class Counter {
3      private final int count;
4      Counter(int c) { this.count = c; }
5
6      Counter increment() {
7          return new Counter(count + 1); // new object each time
8      }
9      int value() { return count; }
10 }
11 // Thread A and B only READ — nothing to synchronize!
```

This is exactly why `String` is shared safely across threads — nothing can ever change it after creation.

Immutable objects are thread-safe because threads can only read them — there is no write to synchronize.



① `final class` — no subclass

② `private final` fields

③ Set only in constructor

④ No setters — ever

⑤ Return new object on change

JDK EXAMPLES

String · Integer · BigDecimal

LocalDate · LocalTime

List.of() · Map.of()

✗ Mutable field (List/array) inside

✗ No defensive copy in constructor

✗ Getter exposes mutable reference

BENEFITS

Thread-safe — no synchronized

Safe HashMap key — stable hashCode

Cacheable — String pool

Predictable — no hidden mutation